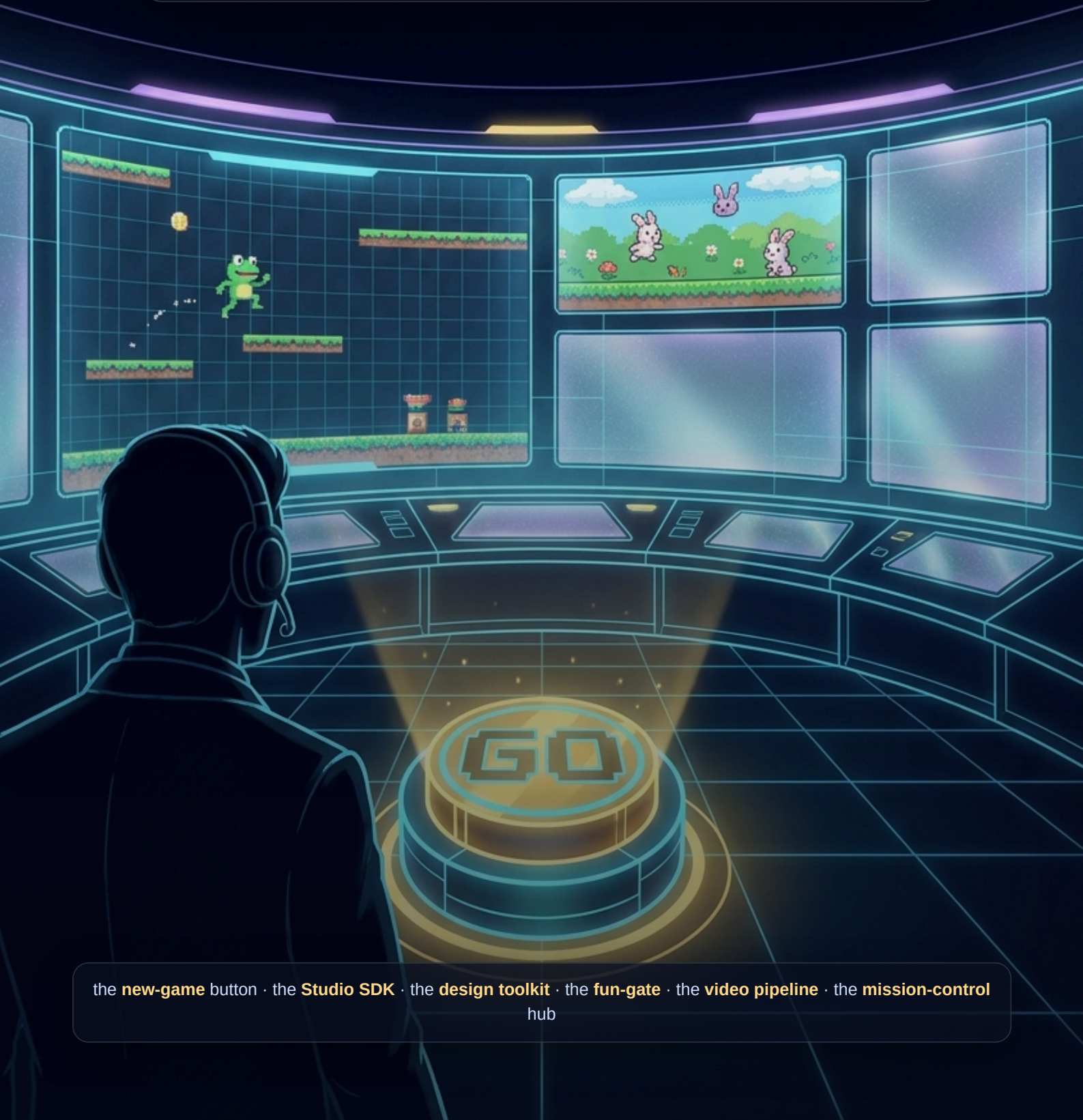


A FIELD GUIDE TO THE ENGINE · BUILT WITH CLAUDE CODE

GAME ENGINE

Skills and Machines — what the AI decides vs what the code does

Eleven games now run on one rig. This edition reads the engine through a single lens: which capabilities live as **skills** (recipes Claude runs by hand) and which have hardened into **programmatic machines** (code that runs itself the same way every time) — and how to keep moving the line.



the **new-game** button · the **Studio SDK** · the **design toolkit** · the **fun-gate** · the **video pipeline** · the **mission-control**
hub

One rig, eleven games, six genres

It started as two games — **the-platformer** and **Jazz** — and a promise: don't rebuild the bottom. It is now a constellation: a brawler, a runner, a vertical climber, a shooter, two car-RTSs, a builder, an axolotl swimmer, an astronaut platformer — all off one engine.

What makes a tenth game cheap is that almost everything is **inherited, not rebuilt**. A new game is mostly a re-skin of a shared machine, plus a few hundred lines of its own level data and identity.

Two materials make up the rig

- **Machines** — deterministic code every game runs verbatim: the `Studio` SDK, the scaffolder, the eval gate, the recorder, the hub.
- **Skills** — prose recipes in `.claude/skills/` that Claude reads and performs: scaffold a game, make a game end-to-end, cut a trailer, rebuild this book.

The health of the engine is the **ratio** between them, and which way it's moving.



The button

The cleanest machine in the repo is the one that makes the next game. It used to be a checklist a human followed. Now it is one command:

```
node scripts/new-game.mjs "My Game" --local --hero "..." --verb "..."
```

- **clone** the de-branded base from `engine/game-template/`
- **rebrand** package, README, a fresh `DIARY.md`
- **write** `GAME_META.json` so the hub shows a rich card
- **create** the GitHub repo, **push**, **register** with the hub

This is the lesson in miniature: a recipe that stopped changing got **frozen into code**. Now it can't drift, runs in seconds, and is the same for game one and game eleven. The whole book is about doing this on purpose, everywhere it's safe.



Recipes and machines

Every capability in this system lives in one of two forms, and they behave very differently.

A skill

PROSE · RUN BY CLAUDE

- A recipe in `SKILL.md` the model reads and performs.
- **Flexible** — adapts to a new genre, a weird game, a fresh idea.
- **Re-judged every run** — so it can drift, vary, or mis-step.
- Costs model time; quality tracks the model.

A machine

CODE · RUNS ITSELF

- A tool/script/SDK function with a fixed contract.
- **Deterministic** — identical inputs, identical output, forever.
- **Testable & fast** — an eval can guard it; it runs in seconds.
- Inherited verbatim by every game; no judgment spent.

Skills are where a capability **lives while it's still being figured out**. Machines are where it goes **once the recipe stops surprising you**. The art is knowing which is which — and moving capabilities across the line as they mature.

Where the line sits today

A faithful map of the engine as it stands — every major capability sorted by its current form.

Machines today

PROGRAMMATIC · IN CODE

- **Studio SDK** — `engine/sdk/studio.js`: Materials, Textures, Toon rigs, Brawl, RNG, Juice, Audio, Cam, the harness.
- **The gate** — per-game `eval.mjs`: determinism + win + non-black, both renderers.
- **The fun model** — `Studio.Brawl.fun` + the platformer `feelmodel` (interest curve → FUN), scored.
- **The design toolkit** — `tools/lib/*` (85 tests): mechanics · enemies · difficulty · reach · feelmodel · fun-max · builders · story · cast · narrative · parity.
- **Scaffolder** — `new-game.mjs` (rich base).
Recorder/shorts/Hub. Headless sim — `Studio._headless`, ~70× scans.

Skills today

RECIPES · RUN BY CLAUDE

- **new-game** — drive the scaffolder, then finish the deploy.
- **make-game** — the whole concept → shipped arc, by hand, stage by stage.
- **trailer** — assemble a montage across games.
- **book** — author + rebuild this PDF.
- *and the judgment that can't be code*: the concept, art direction, what a level should feel like, whether it's fun.

Even **level & story design** have now largely crossed into code (the design toolkit). What remains a skill is the **creative top** — the concept, the art direction, the feel — and the orchestration that strings the machines together (a finer 19-stage pipeline, guarded by the parity gate).

Forging skills into machines

The engine matures by one repeated move: when a step in a skill stops needing judgment, harden it into a tool. Things that have already crossed:

WAS SKILL “follow the scaffold checklist” →

`new-game.mjs` **MACHINE**

WAS SKILL “record each level, mux music, upload” →

`record · make-shorts · host-shorts` **MACHINE**

WAS SKILL “judge if it’s fun” →

`Studio.Brawl.fun` (scored) **MACHINE**

WAS SKILL “scan seeds for a good match” →

`?r=headless` + scan loop **MACHINE**

WAS SKILL “design a fun level · write the world” →

`lib/{feelmodel, funmax, builders, story, cast}` **MACHINE**

WAS SKILL “eyeball the art & richness” →

`full-art.mjs · parity.mjs` gate **MACHINE**

What should cross next

- **The make-game orchestration** → a `ship-game.mjs` that runs the deterministic stages (scaffold · design · gate · deploy · record · host · hub-wire) as one pipeline; the skill keeps only the creative top.
- **A multi-stomp autopilot** → so the gate can certify a `Studio.Boss` finale 0-death (the runtime + drop-wall contract already ship).
- **Diary · screenshots · registration** → a `publish.mjs` that captures stills, writes the scorecard, and registers the hub card.

The design toolkit

The deepest skill of all — judging a level, writing a world, feeling whether a game is *fun* — used to live entirely in the model's head. Now most of its **mechanics** are machines: one tested library, `tools/lib/*.mjs` (85 unit tests), the model reaches for instead of re-deriving each time.

Read a level & score it

- **elements** — every primitive's interest weight + MDA feeling + placement rule.
- **feelmodel** — predicts the per-window interest curve → **FUN** (engagement · dynamics · arc · flow), instantly.
- **reach** — geometric reachability (which coins are stranded) · **difficulty** — the rising arc (calm open, late climax).

Build & improve it

- **builders** — gate-safe campaigns by construction (the autopilot wins 0-death) · **evolve** — edit operators + a critic.
- **funmax** — hill-climbs FUN under a 0-death safety proxy, rolling back any edit that regresses · **Studio.Mechanics** — conveyor/dashpad/crumble/one-way/fields, autopilot-transparent + telegraphed.

Give it a world

- **story** (premise · named antagonist · per-world beats) → **cast** (a named roster, each a role + personality) → **campaign** (WORLDS + arc) → **narrative** (every primitive mapped to the fiction).
- **parity** — fails the build if a game lacks a menu, mobile, touch, real art, ≥3 mechanics, ≥2 enemy kinds, or a story.

The model still owns the *concept* and the *feel*. The labour of measuring, placing, and proving is now deterministic — and an eval guards every piece.

What a new game stands on

Bottom-up. The lower tiers are almost entirely **machine**; the higher you go, the more **skill** (judgment) is mixed in.

Publish & ship · machine

Deterministic recorder → vertical shorts → GitHub-hosted, hub-proxied feed; Railway deploy on push. YouTube optional.

Feedback loop · machine

In-game note → `/api/notes` → GitHub issue on session start → fix → `DIARY.md` → the hub.

Evaluation — the AI playtester · machine

The win+fun gate, deterministic on both renderers; headless seed-scan bakes the highest-fun match into level data.

Design toolkit — the new machines · machine

`tools/lib/*` (85 tests): feel model · reachability · difficulty · fun-max · builders · story · cast · campaign · narrative · parity. Measures, places, and proves what was once pure judgment.

Content & authoring · skill + machine

Gemini art & Lyria music (judgment-led, tool-run); the level builder & `?level=N` contract.

Engine — Studio SDK on Phaser 4 · machine

Toon rigs, archetypes (Brawl...), Materials, Juice, the deterministic stepper + observability harness.

Platform · Foundations · machine

Express · volume KV store · `gemini.js` · Lyria · Railway — and the-harness method beneath it all.

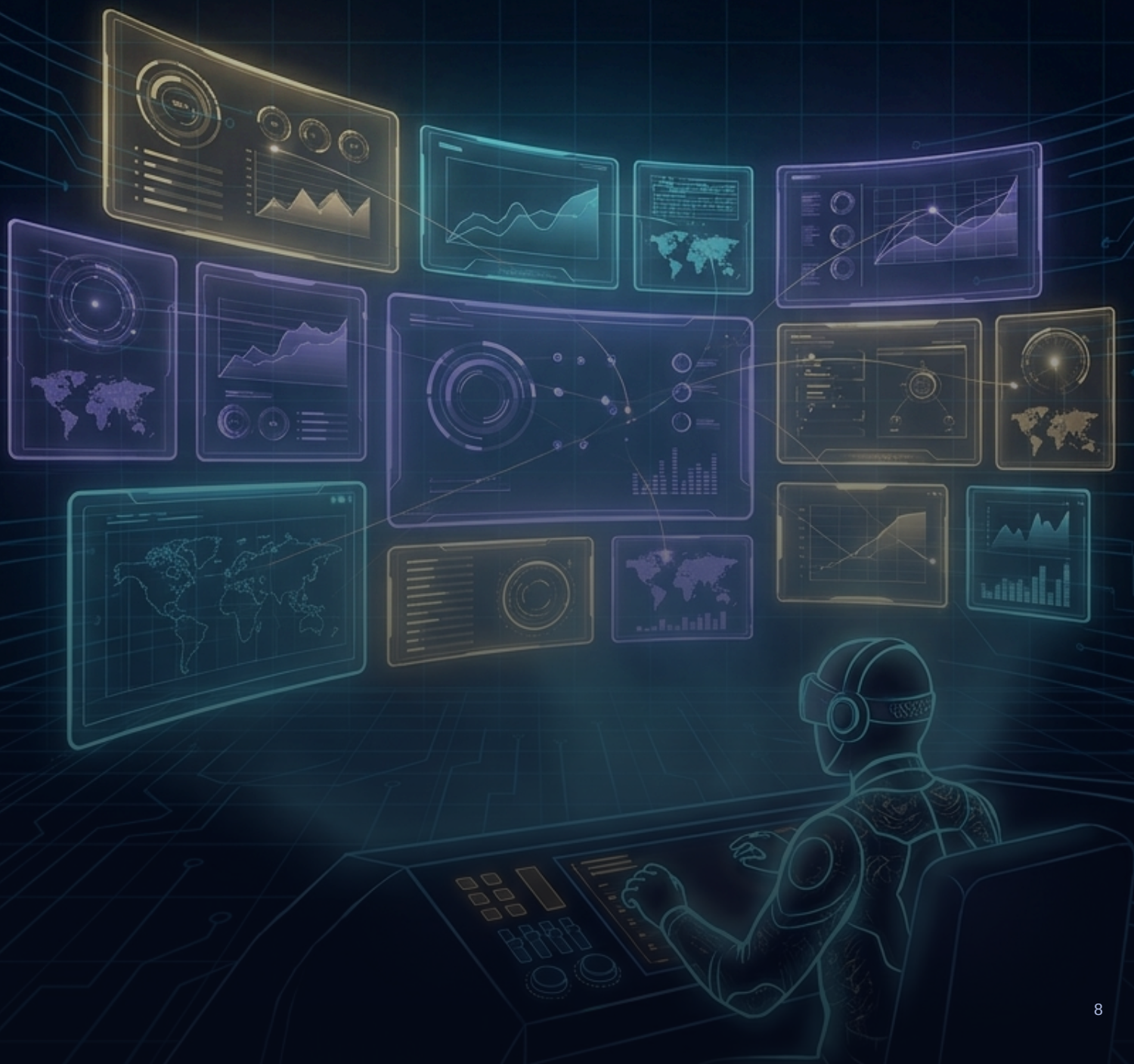
Mission control

One tier sits above every game and runs none of them — it **observes**. Because each game exposes the same surface, the hub pulls them all onto one board.

- `/api/meta` — hero, verb, worlds, level count, scorecard, videos, shorts
- `/api/diary` — the build log, parsed into entries
- `/api/notes` — live playtest feedback, open vs. closed

A storefront, not a spreadsheet

The board is now visual: a gameplay-cover card per game with one coherent button system, a deep-detail modal, and a vertical **shorts feed** — 23 highlight reels streamed same-origin through the hub's video proxy so private-repo assets play everywhere. Pull-model throughout: a game that isn't deployed still shows up from its GitHub repo.



From "I felt something" to "it's fixed"

The reason to push mechanics into machines is to free the skill — Claude's judgment — for the part only it can do: feeling whether a game is good and changing it until it is.

- **Feel it** — pause in-game, pin the spot, leave a note.
- **File it** — each note becomes a GitHub issue on session start.
- **Fix it** — work the issues; the gate guards every change.
- **Record it** — the why goes in the diary; the hub shows it.

The owner gives a concept; the machines do the deterministic labour; the skill spends its judgment on art, feel, and fun. Every capability moved into code is one less thing to re-decide — and one more thing that can't break twice.



Animation that reads, a world that breathes

A character only feels alive if its legs actually **alternate** when it runs and it looks like the **same** creature whether it idles or jumps. Two upgrades made this repeatable for every game, not a hand-tuned one-off.

1 · Characters are generated as sprite SHEETS

Per-pose generation can't keep a run cycle consistent — it returns two “stride” frames with the *same* leg forward, which animates as a **hop**. So `tools/art-sprites.mjs` asks nano-banana-pro for the whole **6-frame cycle in ONE image** (legs scissor across the frames), plus an action grid, chroma-keys them, and slices frames by **connected-component labelling**. The action sheet is conditioned on the cycle sheet, so idle/jump/run are one coherent clay model.

2 · It's validated, and it self-corrects

CHECK manifest in-grid · frames non-blank



no black-box frames **GATE**

CHECK cycle frames move (motion $\geq 3\%$)



never frozen **GATE**

CHECK contact frames differ (alt $\geq 10\%$)



legs scissor, not hop **GATE**

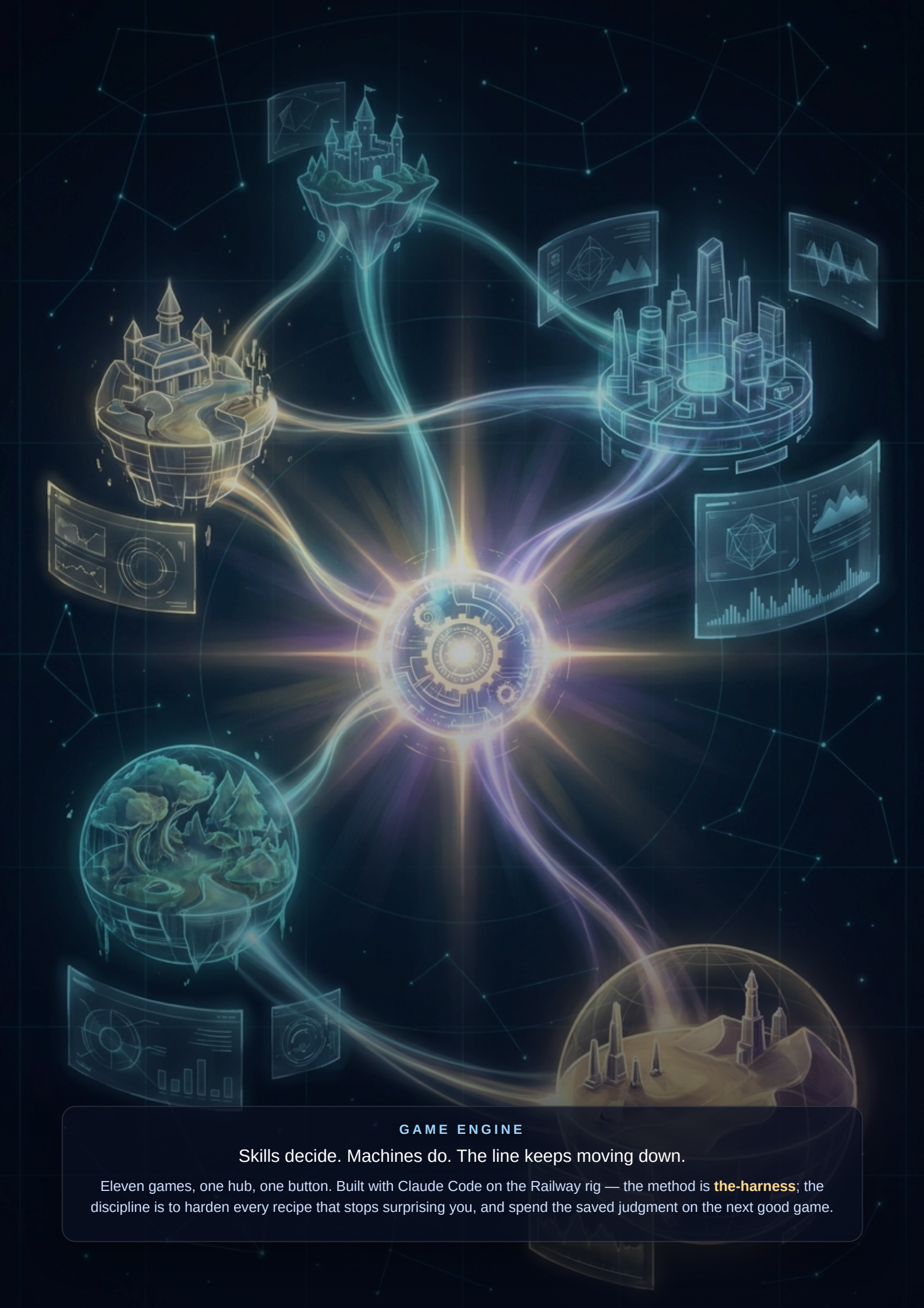
`tools/validate-sprites.mjs` runs after every generation and exits non-zero on failure; a weak cycle **auto-retries** (cache-busted, stronger nudge) up to 3×. Proven live: one critter came out a 7% hop and auto-corrected to 44%. `docs/SPRITES.md` is the written recipe.

3 · A world that moves — `Studio.Atmosphere`

Self-drifting parallax cloud bands (tile-scrolled) + floating motes — petals, snow, dust, bubbles, fireflies — themed per world and fully procedural (it bakes its own textures). The world breathes even when you stand still; purely visual, so it never touches the gate.

4 · The owner's notes close the loop

A note left in-game (Pause → 📝) now **auto-becomes a GitHub issue** (server-side, on a persistent volume so a redeploy never loses it), and a new `add-character` skill turns “I want a grumpy marshmallow” into a validated, animated, wired-in actor. Describe it → it runs.



GAME ENGINE

Skills decide. Machines do. The line keeps moving down.

Eleven games, one hub, one button. Built with Claude Code on the Railway rig — the method is **the-harness**; the discipline is to harden every recipe that stops surprising you, and spend the saved judgment on the next good game.